

CHARACTER CAMEO PJ

Codex初心者向け 開発マニュアル

GitHub と Web 開発にまだ慣れていない人向けの手引きです。このプロジェクトで「新しいキャラを追加する」と「既存ページを直す」を、Codex と一緒に安全に進めるための最短ルートをまとめています。

対象: GitHub初心者 / Pull Request初心者 / Web制作初心者 / Codexをこれから日常的に使いたい人

まだ何も持っていない状態から始める

いちばんやさしい始め方は、**GitHub Desktop** でこのプロジェクトを自分のPCに持ってくる方法です。まずは下の通りにやれば大丈夫です。

- 1 GitHub Desktop を入れる
- 2 GitHub にログインする
- 3 このプロジェクトの GitHub ページを開く
- 4  **Code** → **Open with GitHub Desktop** を押す
- 5 保存場所を聞かれたら、分かりやすい場所を選んで **Clone** を押す
- 6 PCにできた **CharactorCameoPJ** フォルダを Codex で開く

用語をひとことで言うと: GitHub は「置き場所」、GitHub Desktop は「持ってくるアプリ」、Clone は「自分のPCにコピーして作業できるようにすること」です。

押す場所が分からない人向け

1. GitHub Desktop を入れる

<https://desktop.github.com/> を開いてインストールします。難しい設定は後回しで大丈夫です。

2. GitHub に入る

GitHub Desktop を開くとログイン画面が出ます。普段使う GitHub アカウントで入ってください。

3. プロジェクトを開く

ブラウザでこのリポジトリを開き、緑や黒の **Code** ボタンを押します。

4. 自分のPCへ持ってくる

Open with GitHub Desktop が出たらそれを押し、保存先を選んで **Clone** します。

Clone できたあとの最初の一言

ここまで終わったら、Codex には次のように頼めば始めやすいです。

このフォルダを初心者モードで見てください。
いまから何をするかを毎回短く説明しながら進めてください。
まずはこのプロジェクトがちゃんと開けているか確認して、
必要なら新しいブランチを作って作業を始めてください。

「ブランチ」は本番を壊さないための作業用コピーです。今は「安全のために毎回作るもの」くらいで十分です。

まず知っておく4語

ブランチ

本番の **main** を壊さずに作業するための「作業用コピー」です。基本は毎回新しく作ります。

コミット

「この変更をひと区切りで保存する」という記録です。セーブポイントだと思えば十分です。

Push

自分の作業ブランチを GitHub に送る操作です。まだ本番公開ではありません。

Pull Request

「この変更を `main` に入れてよいですか」と確認する提出箱です。通常は Draft で作れば安全です。

一番安全な流れ

- 1 Codex にやりたいことを日本語で頼む
- 2 Codex が新しいブランチを切る
- 3 Codex がファイル編集と `npm.cmd run build` / `npm.cmd run check` を行う
- 4 内容を確認して、問題なければ「PR」と頼む
- 5 GitHub 上で Draft PR を確認する
- 6 レビュー後に merge する

初心者向けの結論: 自分で最初から Git コマンドを覚えなくても大丈夫です。まずは「何を変えたいか」を Codex に伝え、最後に PR を作ってもらう運用で十分進められます。

作業前の準備

- 作業フォルダ: `CharactorCameoPJ` を開いておく
- 確認コマンド: `npm.cmd run build` と `npm.cmd run check`
- ローカル確認: `npm.cmd run dev` を使い、表示された `http://127.0.0.1:{port}/` を開く

- 原則: `dist/` は直接編集しない。正本は `content/` と `docs/`

パターン1: 新しいキャラを追加する

まず Codex に「素材一式がある」ことを伝えます。画像や JSON があるなら、そのフォルダパスごと渡すと進めやすいです。

このフォルダのデータを取り込んで、新しいID `my-character` でページを作って。

足りない情報は推測せず、未定義のままにして。

`build` と `check` まで回して。

Codex がやること

- `content/characters/{characterId}/character.json` を作る
- 画像があれば `assets/` に配置する
- 迷う情報を `content/inbox/{characterId}/README.md` に残す
- `npm.cmd run build` と `npm.cmd run check` を回す

作業後に見る場所

- `dist/{characterId}/index.html`
- `dist/{characterId}/assets/generated/ogp.png`
- `dist/prompts/{characterId}/`

パターン2: 既存ページを編集する

変えたい見た目や文言を、そのまま自然文で頼んで大丈夫です。スクリーンショットがあるとさらに伝わりやすいです。

スマホでルートページを見ると見出しが右に切れる。

横スクロールも出るので直して。

新しいブランチで始めて、`build` と `check` までやって。

`shizumi-oto` のページにバナーが出ていない。

`logo-source.png` を暫定バナーとして使って。PR までお願い。

Codex に頼む時のコツ

- 「どこで」「何が」「どう困るか」を書く
- スクリーンショットや URL を添える
- 最後に `build` と `check` まで と入れる

初心者モード

このプロジェクトでは、アプリの機能としての専用モードより、最初に Codex へ貼るスタートアッププロンプトとして運用するのが一番実用的です。下の文をそのまま使えます。

このスレッドでは初心者モードで進めてください。

- GitHubやWeb開発の用語は、毎回ひとこと補足してください。
- 専門用語を使う時は、小学生向けくらいの言い換えも一緒に入れてください。
- いま何をするかを、作業前に1~2文で説明してください。
- 私がまだ何も分かっていない前提で、押す場所や見る場所もできるだけ具体的に言ってください。
- 危ない操作や後戻りしにくい操作の前では、必ず一度止まって確認してください。
- 変更後は、何を直したか・どこを見ればいいかを短く教えてください。
- 私が「PR」と言うまでは、`commit` / `push` / PR作成は勝手に進めないでください。
- なるべく専門用語を減らして、手順を小さく区切って進めてください。

この文は別ファイルにもしてあります: `docs/codex-beginner-mode-prompt.txt`

困った時の見方

PR に `push` したはずなのに反映されない

その PR がすでに `merged` / `closed` の可能性があります。新しいブランチと新しい PR が必要です。

ページを直したのに公開サイトが変わらない

まだ PR が merge されていないか、GitHub Pages の反映待ちのことがあります。まず PR 状態を見ます。

日本語が文字化けして見える

PowerShell の表示だけ崩れている場合があります。判断は `npm.cmd run check` とブラウザ表示を優先します。

何を頼めばいいかわからない

「このファイルのここを直したい」「この URL の見た目が変」「この素材からページを作って」で十分です。

おすすめしない操作

- `main` に直接 push する
- `dist/` を直接手で編集する
- PR が merge 済みなのに同じブランチへ追記して「同じ PR に入るはず」と考える
- 未定義の設定を想像で埋める
- 文字化けして見えた文字列を、そのまま JSON に貼り戻す

最初の一言テンプレート

このプロジェクトを初心者モードで手伝ってください。
まず新しいブランチを作ってから始めて、
作業前に「今から何をするか」を短く説明してください。
最後は `build / check` の結果まで教えてください。

更新日: 2026-05-12 / Character Cameo PJ 用